Le Gérant de la boutique : les vendeurs

SQL : WHERE, jointure, GROUP BY, HAVING et sous-requêtes

Mise en situation

Tu es le gérant de la boutique. Chaque soir, le caissier te remet la liste des ventes, avec le nom du vendeur qui a réalisé chaque vente. Ta mission : savoir quels vendeurs sont les plus performants.

Les clauses en une phrase

- **Jointure** (WHERE table1.x = table2.x) : on recolle deux tables en ne gardant que les lignes qui ont le même identifiant.
- **WHERE** : le vendeur qui trie les articles avant de les ranger. Il filtre des **lignes**, avant tout regroupement.
- **GROUP BY** : on range les lignes en **paquets** (un paquet par vendeur, par catégorie…).
- **HAVING** : le gérant qui fait le bilan. Il filtre des **paquets** déjà formés.
- **Sous-requête** : une question posée dans une question. On résout la boîte intérieure d'abord, puis l'extérieure.

*Le chiffre d'affaires (CA) d'une ligne est : quantite * prix_unitaire.*

Fiche méthode : construire une requête en 6 étapes

Une requête est une chaîne de montage : le tableau entre, chaque clause le transforme. Pour chaque énoncé, réponds aux 6 questions dans l'ordre.

Étape	Question à se poser	Clause	Mot repère
1	Dans quelle(s) table(s) sont les données ?	FROM	« de la boutique »
2	Les infos sont dans 2 tables ? Quelle colonne les relie ?	WHERE (jointure)	« avec le nom du… »
3	Faut-il écarter des lignes avant de calculer ?	WHERE ... AND	« seulement les articles… »
4	Le résultat est-il demandé « par … » ?	GROUP BY	« par vendeur », « chaque »
5	Y a-t-il une condition sur un total, un nombre, une moyenne ?	HAVING	« dont le CA dépasse… »
6	Que faut-il afficher ?	SELECT	colonnes du GROUP BY + SUM, COUNT…

Gabarit à compléter (on barre les lignes inutiles) :

```
SELECT   ..............................
FROM     ..............................
WHERE    ............... (jointure)
  AND    ............... (filtre des lignes)
GROUP BY ..............................
HAVING   ..............................
```

Règle : la condition porte-t-elle sur une ligne ? Alors WHERE.

Sur un groupe (total, nombre) ? Alors HAVING.

Les tables

Table VENDEUR

id_vendeur	nom
1	Sana
2	Karim
3	Lina
4	Walid

Table VENTE (id_vendeur est la clé étrangère vers VENDEUR)

id	article	categorie	quantite	prix_unitaire	id_vendeur
1	T-shirt	Vêtements	10	25	1
2	Jean	Vêtements	4	60	2
3	Robe	Vêtements	3	90	1
4	Sac	Accessoires	5	40	3
5	Ceinture	Accessoires	6	20	2
6	Écharpe	Accessoires	2	15	3
7	Baskets	Chaussures	3	100	1
8	Sandales	Chaussures	8	35	2
9	Bottines	Chaussures	2	120	3
10	Casquette	Accessoires	7	18	2
11	Chemise	Vêtements	5	45	1
12	Parfum	Parfumerie	1	150	3

Partie A : sur papier (sans SQL)

Avec la table VENTE, calcule le CA de chaque vendeur. Relie d'abord chaque vente à son vendeur grâce à id_vendeur, puis additionne.

Vendeur	Nombre de ventes	CA total
Sana		
Karim		
Lina		
Walid		

Partie B : Caisse ou Bilan ?

Écris W si la condition va dans le WHERE, ou H si elle va dans le HAVING.

Phrase	W ou H ?
a) Seulement les articles dont le prix unitaire dépasse 50.	
b) Seulement les vendeurs dont le CA dépasse 700.	
c) Seulement les ventes de la catégorie Chaussures.	
d) Seulement les vendeurs ayant réalisé au moins 4 ventes.	

Partie C : le film du tableau

Énoncé : afficher, pour chaque vendeur, le CA des articles dont le prix unitaire est 30 ou plus, seulement si ce CA dépasse 700. Pour chaque clause, indique combien de lignes (ou de groupes) contient le tableau après cette étape.

Étape	Clause	Ce qu'on fait	Lignes / groupes
1	`FROM VENDEUR ve, VENTE v`	toutes les combinaisons	
2	`WHERE ve.id_vendeur = v.id_vendeur`	on recolle	
3	`AND v.prix_unitaire >= 30`	on retire des lignes	
4	`GROUP BY ve.id_vendeur, ve.nom`	on fait des paquets	
5	`HAVING SUM(v.quantite * v.prix_unitaire) > 700`	on jette des paquets	
6	`SELECT ve.nom, SUM(v.quantite * v.prix_unitaire)`	on affiche	

Indice : à l'étape 1, chaque vendeur (4) est associé à chaque vente (12).

Partie D : les missions SQL (avec une jointure)

Mission 1 : le registre. Afficher, pour chaque vente, l'article, sa catégorie et le nom du vendeur.

Mission 2 : le classement. Afficher le CA de chaque vendeur (avec son nom).

Mission 3 : les vendeurs performants. Afficher les vendeurs dont le CA dépasse 700.

Mission 4 : les polyvalents. Afficher les vendeurs qui ont vendu dans au moins 3 catégories différentes. Indice : COUNT(DISTINCT ...).

Mission 5 : le tri du vendeur. On ignore les articles dont le prix unitaire est inférieur à 40. Parmi les ventes restantes, afficher les vendeurs dont le CA dépasse 500.

Partie E : les mêmes questions avec des sous-requêtes

Résous de l'intérieur vers l'extérieur : écris d'abord la boîte intérieure et teste-la seule.

E1 (sous-requête dans le SELECT). Afficher le nom de chaque vendeur et son CA. Que remarques-tu pour Walid ?

E2 (sous-requête avec IN). Afficher le nom des vendeurs dont le CA dépasse 700. La boîte intérieure doit renvoyer une liste d'identifiants.

E3 (sous-requête dans le FROM). Refaire la mission 5. La boîte intérieure calcule le CA par vendeur (avec le filtre sur les articles) ; la boîte extérieure garde les CA supérieurs à 500. Il faut un alias pour la table temporaire.

Bonus (NOT IN). Afficher les vendeurs qui n'ont réalisé aucune vente.

Partie F : le caissier distrait (trouve l'erreur)

Chaque requête contient une erreur. Trouve-la et corrige-la.

a)

```
SELECT nom, SUM(quantite * prix_unitaire)
FROM VENDEUR ve, VENTE v
GROUP BY nom;
```

b)

```
SELECT nom, SUM(quantite * prix_unitaire)
FROM VENDEUR ve, VENTE v
WHERE ve.id_vendeur = v.id_vendeur
  AND SUM(quantite * prix_unitaire) > 700
GROUP BY nom;
```

c)

```
SELECT nom, categorie, SUM(quantite * prix_unitaire)
FROM VENDEUR ve, VENTE v
WHERE ve.id_vendeur = v.id_vendeur
GROUP BY nom;
```

d)

```
SELECT nom FROM VENDEUR
WHERE id_vendeur IN (SELECT id_vendeur, nom
                     FROM VENTE
                     GROUP BY id_vendeur
                     HAVING SUM(quantite * prix_unitaire) > 700);
```

e)

```
SELECT nom, ca
FROM VENDEUR ve,
     (SELECT id_vendeur, SUM(quantite * prix_unitaire) AS ca
      FROM VENTE GROUP BY id_vendeur)
WHERE ve.id_vendeur = id_vendeur AND ca > 700;
```

Corrigé

Partie A : résultat attendu

Vendeur	Nombre de ventes	CA total
Sana	4	1045
Karim	4	766
Lina	4	620
Walid	0	0

Vérification : 1045 + 766 + 620 = 2431, le CA total de la boutique.

Partie B

a) W b) H c) W d) H

Une condition sur une ligne (prix, catégorie) va dans WHERE ; une condition sur un total ou un nombre (CA, nombre de ventes) va dans HAVING.

Partie C : le film du tableau

Étape	Clause	Résultat	Lignes / groupes
1	FROM VENDEUR ve, VENTE v	produit cartésien	48 (4 x 12)
2	WHERE ve.id_vendeur = v.id_vendeur	Walid disparaît	12
3	AND v.prix_unitaire >= 30	4 articles retirés	8
4	GROUP BY ve.id_vendeur, ve.nom	un paquet par vendeur	3 groupes
5	HAVING SUM(...) > 700	2 paquets jetés	1 groupe
6	SELECT ve.nom, SUM(...)	affichage	1 ligne

Les 8 lignes restantes à l'étape 3, rangées en paquets à l'étape 4 :

Vendeur	Lignes (article : CA)	Total	Total > 700 ?
Sana	Robe 270, Baskets 300, Chemise 225	795	OUI : gardé
Karim	Jean 240, Sandales 280	520	NON : jeté
Lina	Sac 200, Bottines 240, Parfum 150	590	NON : jeté

Résultat : Sana 795. Les articles retirés par le WHERE sont : T-shirt (25), ceinture (20), écharpe (15), casquette (18).

```
SELECT ve.nom, SUM(v.quantite * v.prix_unitaire) AS ca
FROM VENDEUR ve, VENTE v
WHERE ve.id_vendeur = v.id_vendeur
  AND v.prix_unitaire >= 30
GROUP BY ve.id_vendeur, ve.nom
HAVING SUM(v.quantite * v.prix_unitaire) > 700;
```

Partie D : solution 1, avec une jointure

Mission 1

```
SELECT v.article, v.categorie, ve.nom
FROM VENTE v, VENDEUR ve
WHERE v.id_vendeur = ve.id_vendeur;
```

12 lignes. Walid n'apparaît pas : il n'a aucune vente.

Mission 2

```
SELECT ve.nom, SUM(v.quantite * v.prix_unitaire) AS ca
FROM VENDEUR ve, VENTE v
WHERE ve.id_vendeur = v.id_vendeur
GROUP BY ve.id_vendeur, ve.nom;
```

Sana 1045 | Karim 766 | Lina 620

Mission 3

```
SELECT ve.nom, SUM(v.quantite * v.prix_unitaire) AS ca
FROM VENDEUR ve, VENTE v
WHERE ve.id_vendeur = v.id_vendeur
GROUP BY ve.id_vendeur, ve.nom
HAVING SUM(v.quantite * v.prix_unitaire) > 700;
```

Sana 1045 | Karim 766

Mission 4

```
SELECT ve.nom, COUNT(DISTINCT v.categorie) AS nb_rayons
FROM VENDEUR ve, VENTE v
WHERE ve.id_vendeur = v.id_vendeur
GROUP BY ve.id_vendeur, ve.nom
HAVING COUNT(DISTINCT v.categorie) >= 3;
```

Karim 3 | Lina 3 (Sana n'a vendu que dans 2 rayons : Vêtements et Chaussures).

Mission 5

```
SELECT ve.nom, SUM(v.quantite * v.prix_unitaire) AS ca
FROM VENDEUR ve, VENTE v
WHERE ve.id_vendeur = v.id_vendeur
  AND v.prix_unitaire >= 40
GROUP BY ve.id_vendeur, ve.nom
HAVING SUM(v.quantite * v.prix_unitaire) > 500;
```

Sana 795 | Lina 590. Karim tombe à 240 : le WHERE retire la ceinture (20), les sandales (35) et la casquette (18), il ne lui reste que le jean.

Solution 2, avec des sous-requêtes

Idée : on ne recolle plus les tables. On résout la boîte intérieure d'abord (seule, on peut la tester), puis la boîte extérieure l'utilise comme une donnée.

Type	Où elle se place	Renvoie	Usage
Valeur	dans le SELECT, ou après > ou =	1 valeur	calculer une colonne, comparer à une moyenne
Liste	après IN / NOT IN	1 colonne	filtrer sur un ensemble
Table temporaire	dans le FROM (alias obligatoire)	une table	remplacer HAVING par un simple WHERE

E1 : sous-requête dans le SELECT

```
SELECT nom,
       (SELECT SUM(quantite * prix_unitaire) FROM VENTE
        WHERE VENTE.id_vendeur = VENDEUR.id_vendeur) AS ca
FROM VENDEUR;
```

Sana 1045 | Karim 766 | Lina 620 | Walid NULL. Contrairement à la jointure, Walid apparaît, avec un CA vide.

E2 : sous-requête avec IN (mission 3)

```
SELECT nom
FROM VENDEUR
WHERE id_vendeur IN (SELECT id_vendeur
                     FROM VENTE
                     GROUP BY id_vendeur
                     HAVING SUM(quantite * prix_unitaire) > 700);
```

Sana, Karim. Boîte intérieure seule : (1, 2). Boîte extérieure : WHERE id_vendeur IN (1, 2). Cette version n'affiche pas le CA.

E2 bis : sous-requête dans le FROM (affiche le CA)

```
SELECT ve.nom, t.ca
FROM VENDEUR ve,
     (SELECT id_vendeur, SUM(quantite * prix_unitaire) AS ca
      FROM VENTE
      GROUP BY id_vendeur) t
WHERE ve.id_vendeur = t.id_vendeur
  AND t.ca > 700;
```

Sana 1045 | Karim 766. Le HAVING a disparu : le CA est devenu une colonne de la table temporaire t, donc un simple WHERE suffit. HAVING = WHERE appliqué à des groupes déjà calculés.

E3 : mission 5 avec une sous-requête dans le FROM

```
SELECT ve.nom, t.ca
FROM VENDEUR ve,
     (SELECT id_vendeur, SUM(quantite * prix_unitaire) AS ca
      FROM VENTE
      WHERE prix_unitaire >= 40
      GROUP BY id_vendeur) t
WHERE ve.id_vendeur = t.id_vendeur
  AND t.ca > 500;
```

Sana 795 | Lina 590. Le WHERE de la boîte intérieure filtre les lignes ; la condition sur le CA, dans la boîte extérieure, filtre les groupes.

Mission 4 avec IN (polyvalents)

```
SELECT nom
FROM VENDEUR
WHERE id_vendeur IN (SELECT id_vendeur
                     FROM VENTE
                     GROUP BY id_vendeur
                     HAVING COUNT(DISTINCT categorie) >= 3);
```

Karim, Lina.

Bonus : NOT IN

```
SELECT nom
FROM VENDEUR
WHERE id_vendeur NOT IN (SELECT id_vendeur FROM VENTE);
```

Walid. La jointure ne garde que les lignes qui ont un correspondant, donc Walid disparaît de toutes les jointures ; NOT IN permet de le retrouver.

Pour aller plus loin : au-dessus de la moyenne

```
SELECT id_vendeur, SUM(quantite * prix_unitaire) AS ca
FROM VENTE
GROUP BY id_vendeur
HAVING SUM(quantite * prix_unitaire) >
       (SELECT SUM(quantite * prix_unitaire) / COUNT(DISTINCT id_vendeur)
        FROM VENTE);
```

La moyenne par vendeur ayant vendu vaut 810,33 (2431 / 3) : seul le vendeur 1 (Sana, 1045) la dépasse.

Jointure ou sous-requête ?

	Jointure (table1.x = table2.x)	**Sous-requête**
Idée	recoller puis regrouper	calculer d'abord, utiliser ensuite
Colonnes des 2 tables	directement dans le résultat	via le SELECT ou le FROM
Walid (sans vente)	disparaît	présent (E1) ou retrouvé (NOT IN)
Comparer à une moyenne	difficile	naturel

Partie F : erreurs

- **a)** Pas de condition de jointure : produit cartésien, chaque vendeur additionne toutes les ventes. Ajouter WHERE ve.id_vendeur = v.id_vendeur.
- **b)** Un agrégat est interdit dans WHERE : la condition sur SUM(...) va dans HAVING, après le GROUP BY. La condition de jointure reste dans WHERE.
- **c)** categorie n'est ni agrégée ni dans le GROUP BY. La retirer du SELECT, ou l'ajouter au GROUP BY (un total par vendeur et par catégorie).
- **d)** La boîte intérieure d'un IN doit renvoyer une seule colonne : retirer « nom » du SELECT intérieur.
- **e)** La table temporaire n'a pas d'alias, et id_vendeur est ambigu. Ajouter un alias (t) après la parenthèse et écrire ve.id_vendeur = t.id_vendeur AND t.ca > 700.